Fixed-K Speculative Decoding for Energy-Proportional LLM Serving: A Reproducible Empirical Validation

Zanno Jacklin
Independent Researcher, Creepybits

***Abstract*—We present an empirical validation of fixed draft-length ($K{=}5$) speculative decoding using a scout(1B)→target(8B) model pair on consumer GPU hardware. Across three prompt categories spanning low to high output determinism (creative poetry, technical explanation, and code generation), we measure throughput, energy per token, and output fidelity relative to a matched FP16 autoregressive baseline. Speculative decoding achieves up to +78.5% throughput improvement and −59.3% energy reduction per token on code generation, with smaller but consistent gains on less predictable prose (+6.7% / −32.2% on poetry). All results achieve 100% exact-token-match fidelity against the baseline (lossless, greedy decoding) and are independently reproducible from the accompanying open-source repository. We report results transparently, including the deliberate methodological choice to omit KV-caching from both arms of the comparison, which keeps the relative comparison fair but means absolute throughput figures are not representative of a production-optimized server, and including a documented measurement-methodology correction (a warmup-timing bug affecting an earlier version of this result) alongside the closed-loop warmup convergence and residual-drift diagnostics now used to guard against a recurrence.**

***Index Terms*—speculative decoding, energy-efficient inference, large language models, GPU power measurement**

I. Introduction

Autoregressive decoding in large language models (LLMs) is inherently sequential: each output token requires a full forward pass through the target model before the next token can be produced. This serial dependency underutilizes modern GPU compute, which is optimized for parallel throughput. Speculative decoding [1], [2] addresses this by using a smaller, faster *scout* model to propose a short window of candidate tokens, which the larger *target* model then verifies in a single parallel forward pass, accepting the longest correct prefix.

This paper reports a narrow, fully-validated empirical claim: a fixed draft window of $K{=}5$ tokens, paired with an exact (lossless) accept/reject verification scheme, produces measurable throughput and energy improvements on real consumer GPU hardware, with perfect output fidelity relative to standard greedy decoding. We deliberately scope this paper to only the mechanism we have fully validated. A related, more ambitious mechanism under investigation in the parent project — entropy-gated *dynamic* adjustment of the draft window, intended to switch between aggressive and conservative speculation based on local output entropy — is explicitly **not** included here, as it has not yet been shown to outperform this simpler fixed-K approach in single-request testing.

A. Relationship to prior work

Entropy-gated adaptive speculative decoding is an active research area. AdaEDL [3] reports 10–57% improvement over static draft length using entropy-based adaptive drafting. SGLang ships a built-in adaptive speculative-length mechanism using an EMA of accepted draft length. Our own testing of an entropy-gated variant, using the same hardware and methodology as this paper, has not yet demonstrated an improvement over the simpler fixed-K approach reported here; we regard that as an open question requiring further work (e.g. testing under multi-request/batched serving, which published entropy-gating results suggest is where the technique's value proposition may actually lie), not a settled negative result. This paper makes no claims about the entropy-gated mechanism in either direction.

II. Methodology

A. Models and hardware

Scout model: Llama-3.2-1B-Instruct. Target model: Llama-3.1-8B-Instruct. Both loaded in bfloat16. All measurements taken on a single NVIDIA RTX 5090 (Blackwell architecture) GPU, single-request (batch size 1), greedy (argmax) decoding throughout. The results reported here were collected under WSL2, without NVML GPU clock locking (no elevated privileges available in that environment); this is noted for reproducibility, since clock-locked runs would be expected to show lower run-to-run variance.

B. Draft-verify procedure

At each decoding cycle, the scout model autoregressively proposes $K{=}5$ candidate tokens. The target model then performs a single forward pass over the full draft sequence, and its argmax predictions at each position are compared against the corresponding drafted tokens. The longest matching prefix is accepted; at the first mismatch (or after all K tokens match), the target's own prediction is substituted, exactly reproducing what standard greedy decoding would have produced at that position. This verification scheme is lossless by construction: the output token sequence is provably identical, position-for-position, to what running the target model alone would produce, since every emitted token is either a drafted token confirmed by the target's own argmax, or the target's argmax directly.

C. Power and energy measurement

GPU power is sampled via NVML at 100 Hz on a background thread throughout each timed generation window. Energy per token is read from the GPU's onboard hardware energy counter (`nvmlDeviceGetTotalEnergyConsumption`) when available, as it was for every trial reported in this paper, divided by N_{tok}, the number of tokens produced. This is a more direct measurement than integrating sampled instantaneous power readings, which is retained in the accompanying code only as a fallback and cross-check for hardware that lacks the counter.

D. Warmup

Warmup is applied in two layers. First, a one-time *closed-loop thermal warmup* runs real, discarded decoding cycles – alternating across all prompts and both the baseline and speculative conditions – until GPU die temperature stops moving across several consecutive readings *and*, separately, each individual prompt/condition combination's own power reading stops moving relative to its own recent history. The latter distinction matters because baseline and speculative decoding draw genuinely different power by design (see Section III); a convergence check that simply requires consecutive readings to agree with each other, without accounting for which workload produced them, can fail to terminate when the workload alternates between conditions with a real, persistent power gap. An earlier version of this warmup procedure additionally used a warmup burst length shorter than the real trial length, which was found empirically to converge at a lower power/thermal level than the real, longer trial then reached – the first real trial after such a warmup showed the GPU still heating measurably mid-measurement. Both issues are corrected in the procedure used for the results in this paper: warmup burst length matches the real trial length, and convergence is checked per-workload rather than across the pooled, alternating sequence. Second, independently of the closed-loop warmup above, each individual timed trial is still preceded by 5 short untimed forward-pass warmup steps immediately before its own measurement window, avoiding cold-SM effects specific to that trial. This two-layer procedure is applied in all three scripts (`benchmark_ablation.py`, `fp16_baseline.py`, and `speculative_scout.py`), though not from one shared implementation: `benchmark_ablation.py` and `speculative_scout.py` both call the procedure from `bench_common.py`, while `fp16_baseline.py` was rewritten independently and carries its own equivalent implementation of the same warmup and drift-diagnostic logic. The per-workload convergence check in `bench_common.py` is a superset of what `fp16_baseline.py` needs – `fp16_baseline.py` only alternates between prompts under a single (baseline-only) workload, which the simpler pooled convergence check handles correctly, whereas `benchmark_ablation.py` alternates between two workloads (baseline and speculative) with a real, persistent power gap, which is what motivated the per-workload check in the first place.

E. Deliberate exclusion of KV-caching

Both the baseline and speculative decoding paths recompute attention over the full sequence at every step, with no KV-cache reuse. This was a deliberate methodological choice: an earlier iteration of this work applied caching only to the baseline arm, which structurally penalized the speculative path whenever it fell back to single-step generation (a full uncached recompute for a single token is the worst-case cost ratio in the system). Removing caching from both arms restores a fair relative comparison at the cost of both arms being substantially slower in absolute terms than a production server with caching would be. We therefore report *relative* throughput and energy figures as the validated result, and explicitly flag that absolute tok/s numbers in this paper are not production-representative.

F. Fidelity measurement

Fidelity is measured as the exact token-for-token match rate between the baseline's greedy output and the speculative path's output, over the full generated sequence, for each prompt.

G. Prompts

Three prompts were selected to span a range of output determinism:

- **Poem** (low determinism): an open-ended creative writing task (an original Chant Royal poem on a mathematical theme).
- **Physics** (medium determinism): a technical explanation task (semiconductor memory bandwidth and the memory wall).
- **Code** (high determinism): a code-generation task (a Python binary search tree implementation with type annotations), where correct completions are far more constrained token-to-token.

III. Results

A. Fidelity

Aggregate exact-token-match fidelity across all three prompts was 100.0%, confirming the accept/reject implementation is lossless in practice as well as by construction.

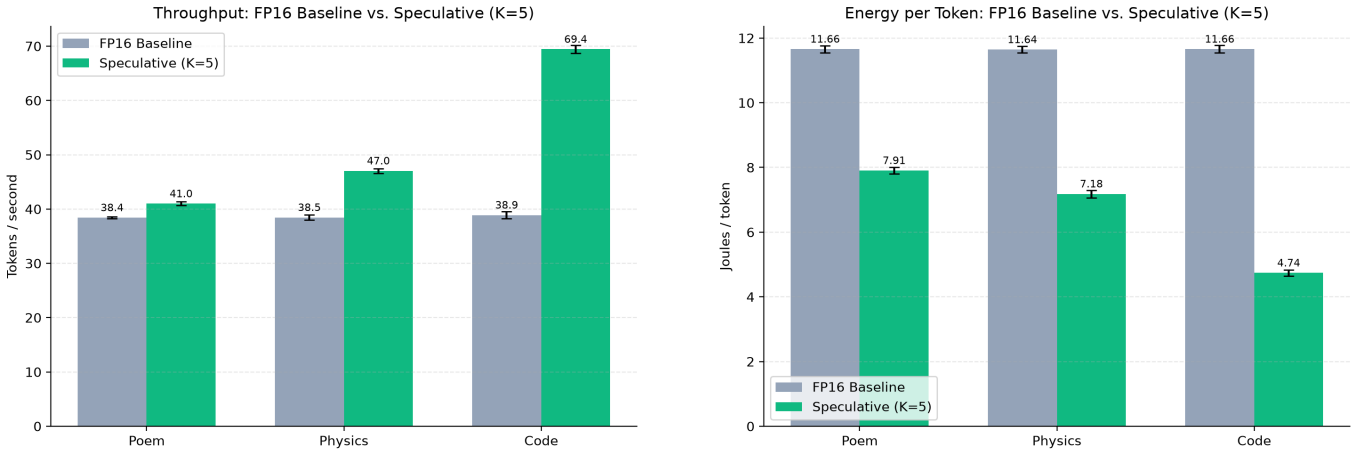
B. Throughput, energy, and power

Table I reports mean $\pm$ standard deviation across $N{=}10$ trials per prompt per condition.

Mean GPU power during speculative decoding (344–363 W) is lower than during FP16 baseline decoding (466–469 W), despite two models being resident in VRAM simultaneously. This is consistent with fewer full 8B-parameter forward passes being required per unit of output as accepted draft batch length grows – each accepted cycle of up to $K{+}1{=}6$ tokens requires only one target forward pass, versus six for the baseline.

TABLE I: Baseline vs. fixed-K=5 speculative decoding, per prompt. N=10 trials/prompt. Collected 2026-09-04; see Section III-C for the measurement-methodology correction this run reflects.

Prompt	Baseline tok/s	Spec. tok/s	Speedup	Baseline J/tok	Spec. J/tok	Energy Δ	Accept %
Poem	38.42 ± 0.18	40.98 ± 0.35	+6.7%	11.66 ± 0.11	7.91 ± 0.10	−32.2%	42.5%
Physics	38.45 ± 0.50	47.03 ± 0.42	+22.3%	11.64 ± 0.11	7.18 ± 0.12	−38.3%	51.7%
Code	38.90 ± 0.67	**69.44 ± 0.73**	**+78.5%**	11.66 ± 0.12	**4.74 ± 0.09**	**−59.3%**	**85.4%**



(a) Throughput

(b) Energy per token

Fig. 1: FP16 baseline vs. fixed-K=5 speculative decoding, per prompt (N=10 trials/prompt, error bars show ±1 std. dev.). Gains are smallest on the least predictable content (Poem) and largest on the most predictable (Code).

C. *Measurement validity: residual drift diagnostics*

After the correction to the warmup procedure described in Section II-D, each run's per-trial telemetry is checked for residual drift: a least-squares fit of power, energy/token, and throughput against chronological trial index within the run. For the run behind Table I, the fitted trend was negligible for every metric, pooled and per-condition ($R^2 \leq 0.09$ in all cases, most $R^2 \approx 0$), and the total drift over the full 60-trial run was small in absolute terms (≈ -2 W against a pooled mean of $\approx$ 410 W). The accompanying code's pass/fail threshold ($\pm 0.5\%$ of the mean) flagged this run regardless (pooled -0.53%; speculative-only -0.58%; baseline-only passed at -0.48%), since that threshold does not account for R^2. We report both numbers here rather than silently relying on the pass/fail flag: a near-zero R^2 alongside a sub-1%-of-mean total change is consistent with ordinary run-to-run measurement noise rather than a systematic thermal trend contaminating the reported means, and we treat the results in Table I as valid on that basis. Separately, the closed-loop warmup itself did not fully converge within its time cap on this specific run (WSL2 GPU power-reporting granularity appears coarser than the convergence tolerance can reliably satisfy); the drift diagnostic above is the safeguard designed for exactly this situation, and it returned a clean result.

D. *Speedup and energy reduction scale with acceptance rate*

Figure 2 shows speedup tracking draft acceptance rate across the three prompts, consistent with the mechanism's expected behavior: more predictable continuations (code) allow the scout to correctly anticipate more tokens per cycle, directly translating to fewer expensive target forward passes.

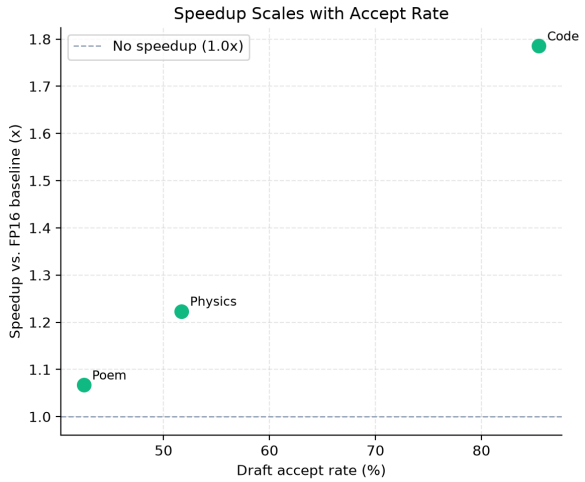


Fig. 2: Speedup vs. FP16 baseline as a function of draft accept rate, across the three prompts. The relationship is monotonic and consistent with the mechanism's expected behavior.

IV. Discussion and Limitations

A. Absolute throughput is not production-representative

As noted in Methodology, neither arm of this comparison uses KV-caching. A production serving stack with proper cache reuse would show substantially higher absolute throughput in both arms; we have not yet validated fixed-K speculative decoding's behavior under a cached, production-style serving loop, and consider this the primary remaining gap before these results can inform a deployment decision.

B. Single-request, single-GPU scope

All results in this paper are single-request (batch size 1) measurements on a single consumer GPU. Behavior under batched, multi-request serving – where the entropy-gated dynamic variant may plausibly outperform this fixed-K approach, per precedent in AdaEDL and SGLang – is untested and out of scope for this paper.

C. Reproducibility

All code, raw telemetry (JSON/CSV), and the scripts used to generate every figure and table in this paper are available at https://github.com/Creepybits/sdsie-fixed-k5-speculative-decoding under an Apache-2.0 license. Reported numbers are directly traceable to `telemetry/ablation_results.json`, generated by `benchmark_ablation.py` in that repository. The closed-loop warmup and drift-diagnostic logic described in Sections II-D and III-C is shared between `benchmark_ablation.py` and `speculative_scout.py` via `bench_common.py`; `fp16_baseline.py` implements the same logic independently (see Section II-D).

V. Conclusion

Fixed-$K{=}5$ speculative decoding, with an exact lossless verification scheme, produces measurable and reproducible throughput and energy gains on real consumer GPU hardware, scaling with output predictability, with zero loss of output fidelity. We report this as a validated, narrow result, distinct from and not dependent on the more experimental entropy-gated dynamic mechanism under investigation elsewhere in the parent project.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023.

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating Large Language Model Decoding with Speculative Sampling," arXiv:2302.01318, 2023.

[3] Qualcomm AI Research, "AdaEDL: Adaptive Entropy-based Dynamic Layer for Efficient Speculative Decoding," arXiv:2410.18351, 2024.